PROTOTYPING

INTRODUCTION

Prototyping in the fourth stage of design thinking. It involves taking the best ideas from brainstorming sessions to a higher degree of resolution and detail by building a model or prototyping the solution. A model or prototype is not necessarily an object or building; it is some sort of solution or "deliverable." For example, it could be anything from a strategy, an app, story, or experience, to a business model that functions as a demonstration—an "operational prototype"—of the idea.

A prototype can be a paper model, storyboard, wireframe or a cardboard box - it allows you to quickly visualize and identify the best solution among several concepts. It is a way to convey an idea quickly. Ideally, it is recommended that a prototype should be cheap, fast and easy so as to eliminate the pain of cost and effort- should refinement and iteration be needed. This phase of design thinking involves narrowing down all the ideas from brainstorming to those that are the most promising (convergent thinking).

WHY DO WE NEED A PROTOTYPE?

There are two significant goals of creating a model. The first is using the act of creating the model as a tool to develop an idea into a coherent solution. Making things—whether it be a physical three dimensional model or a narrative description of a strategy—is crucial to innovation (and in many circumstances, funding).

The second objective in building a model is to get feedback through testing and critical evaluation. The model, therefore, should be a very close embodiment of a proposed (draft) solution in order to elicit the most constructive comments and critique.

Why is feedback so important? Conversations and critiques help design thinkers to personally reflect on the process. When you have to articulate an idea and respond to criticism you are forced to think through more aspects of an issue, and ideas tend to become more concrete. On the other hand, if you keep something locked inside your head without having a dialogue, the reflection phase is neglected as you charge forward, with potential loss of opportunities and insights.

Other uses of prototype:
1 — Find design issues early
2 — Iterate more quickly on a design concept
3 — Compare design variations quickly
4 — Gather design feedback better
5 — Good presentational tool
6 — Encourage collaboration
7 — Cheap, fast and easy
8 — Gather design feedback better

CASE STUDY

One of the classic examples is the Microsoft Zune mp3 player, and its sequel Zune HD. Launched in 2006, Zune was Microsoft's answer to Apple's iPod. Unfortunately, it had a mixed reception. Some praised its features, while other reviewers criticized its design and bulkiness. In 2009, thousands of Zune players froze. Microsoft said it was due to software problems with its internal clock. Zune was finally discontinued after 2010.

In Daniel Ling's opinion, this was a classic case of a company pressured to launch its product/ service hastily and hoping for the best, without adequate prototyping or testing. The end-users got "burnt" and complained, and the design team had to do the painful, expensive and demanding task of reworking and redesigning.

One of the key skills of the design thinker is to ensure that we prototype and test every solution early and cheaply so as to discover any problems or issues before the product is rolled out. It saves cost and effort throughout the process and minimizes chances for errors.

TYPES OF PROTOTYPES

There are four types of prototypes:

1. Physical Prototypes
2. Analytical Prototypes
3. Comprehensive Prototypes
4. Focused Prototypes

Physical Prototypes

Physical prototypes are tangible artifacts created to approximate the product. Aspects of the product of interest to the development team are actually built into an artifact for testing and experimentation. Examples of physical prototypes include models that look and feel like the product, proof-of-concept prototypes used to test an idea quickly, and experimental hardware used to validate the functionality of a product.

Analytical Prototypes

Analytical prototypes represent the product in a nontangible, usually mathematical or visual, manner. Interesting aspects of the product are analyzed, rather than built. Examples of analytical prototypes include computer simulations, systems of equations encoded within a spreadsheet, and computer models of three dimensional geometry.

Comprehensive Prototypes

Comprehensive prototypes implement most, if not all, of the attributes of a product. A comprehensive prototype corresponds closely to the everyday use of the word *prototype*, in that it is a full-scale, fully operational version of the product. An example of a comprehensive prototype is one given to customers in order to identify any remaining design flaws before committing to production.

Focused Prototypes

Focused prototypes implement one, or a few, of the attributes of a product. Examples of focused prototypes include foam models to explore the form of a product and hand-built circuit boards

to investigate the electronic performance of a product design. A common practice is to use two or more focused prototypes together to investigate the overall performance of a product. One of these prototypes is often a "looks-like" prototype, and the other is a "works-like" prototype. By building two separate focused prototypes, the team may be able to answer its questions much earlier than if it had to create one integrated, comprehensive prototype.

Note that focused prototypes can be either physical or analytical, but that for tangible manufactured products, fully comprehensive prototypes must generally be physical. Prototypes sometimes contain a combination of analytical and physical elements.

Fidelity for Prototypes

Choosing a fidelity level is a critical part of creating a prototype. Fidelity means how closely the prototype looks and acts like the finished product. The proper fidelity level will focus the feedback you receive on the proper aspect of the design, so select your fidelity based on your goal for the prototype. Fidelity has varying levels (low, mid, and high, as well as mixed) and five dimensions (visual, breadth, depth, interactivity, and data model). It takes time and practice to learn which fidelity will enable you to get the feedback you need, but there are a few best practices for choosing.

The prototyping process usually benefits from starting with a low fidelity and slowly increasing the fidelity level until most of your assumptions are tested and either proved or fixed. You'll find that you make more prototypes earlier in the process, and fewer as your idea becomes more refined. It's necessary to be flexible and to decide which fidelity is right for each assumption you're testing along the way. If a prototype's fidelity level is too high, the user will subconsciously believe that the design is "finished" and will only give feedback on polishing areas instead of the broad concepts. If a prototype's fidelity is too low, the user might not understand the context and get lost in the generalities. There's also a balance between the time and effort it takes to make the prototype and the value you'll get from testing at that specific fidelity (Figure 4.1). By choosing the proper fidelity, or by creating a mixed fidelity, for your point in the process and the goal of the prototype, you'll save time and get the proper feedback you need to improve your ideas.

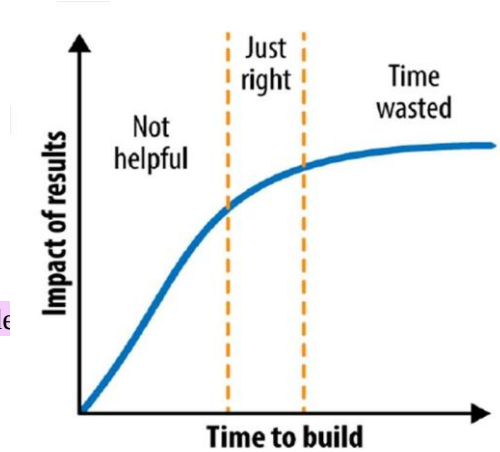


Figure 4.1: You must de prototype based on its

Low Fidelity

Low-fidelity prototypes are best for testing your core concepts, getting over initial fears, thinking through many ideas, and catching potential problems before they get too big to fix. This type of prototype doesn't look like your final product at all; it's in a different medium, at a different size, and is usually not visually designed (although you should be thinking about visual design during this whole process).

It's the easiest and cheapest prototype to make, and doesn't require as much time or skill to complete. Some examples include paper prototypes, circuit building, storyboards, wireframes, mood boards, sketches, and component prototypes. The goal of a low-fidelity prototype is to test basic and big assumptions. With this rough prototype, your user won't waste time giving feedback on the execution and appearance of the interface or device, and rather will focus on the overall use and flow of the product.

For example, when I'm working on the initial information architecture of a site (how a site is organized, what terminology to use for the specific user, and how to group labels in the most intuitive way), I first do a card-sorting activity. I give my user a set of cards that has all of my navigation pages and names on them and ask them to organize the cards in a way that makes sense for her or him. This activity allows me to understand my user's mental model because each individual will organize differently, and my navigation needs to work for all of these different mental models. It takes very little time, material, and effort to make a card-sorting activity, and I don't need an interface to test. I can learn a lot about my users, though, and improve the layout and navigation of my product to better suit them.

Mid-Fidelity

Mid-fidelity prototypes start to look like your final product in at least one dimension. They are a good balance between cost (time or otherwise) and value. Some examples include clickable prototypes, style tiles, Axure prototypes, coded prototypes, and a variety of electronic prototypes.

Mid-fidelity prototypes take longer to make than low-fidelity ones, but you can begin testing more detailed parts of the interactions. The users will have more context in the prototype itself, which will give you more sound results in testing.

Mid-fidelity prototypes are useful for communication with stakeholders who might not have the ability to properly "read" low-fidelity work. You're showing a more refined view of the concept with better context. It's a good balance of the amount of time to make and more detail as to what you're building. With this fidelity level, the stakeholder doesn't have to imagine how the product will look in real life (either in a browser or as a physical object) and they can feel confident in the direction the design work is moving.

High Fidelity

High-fidelity prototypes are the real deal. They are visually designed and are in the final medium of either physical materials or code in a browser. These prototypes have real content and most paths available to click or interact with. Some examples include a highly polished electronic smart object, a coded app, or a fully designed digital experience. High-fidelity prototypes cost more to make in time, skill level, and the software or coding needed to complete

hem. You can make high-fidelity physical prototypes by CNC milling, casting, sewing soft goods, or having circuit boards printed for you.

Toward the end of a software development release, engineering and design teams work together to build high-fidelity coded prototypes in the final product form so that after testing, if the results are positive, they can quickly ship the prototype into production. It's a bit risky because testing might still disprove some assumptions about the product, and the cost of building at high fidelity is high. But it gives one last layer of testing before releasing a product. Each release of a product is an opportunity to improve it, so even after shipping a final product, most Agile teams will continue to improve on the product based on user feedback and backlog of improvements.

	LOW FIDELITY	MID-FIDELITY	HIGH FIDELITY
Pros	Fast, low-skill, cheap, made with materials available around you	More interactive, easier to test, good balance of time and quality	Complete design, including visuals, content, and interactions; can test very detailed interactions
Cons	Limited interactions, harder to test details and full flows, little context for users	More time-intensive, but not fully functional	Very time-intensive, requires skills with software or coding, hard to test large concepts
Use	Exploring and testing high-level concepts like user flows and information architecture; best for making lots of different versions and testing them against each other	User testing specific interactions and guided flows; also better for stakeholder presentations, as these prototypes have more context	User testing very specific interactions and details, final testing of user flows, and presenting final design work to stakeholders

Table 4.1. Pros and cons of each type of fidelity

Summary

Fidelity levels are a vital part of prototyping, and greatly affect the outcome of testing. You must choose the proper fidelity level depending on your goals and what part of the development process you are in.

- Low-fidelity prototypes are fast and cheap to make, and are good for testing high-level concepts.
- Mid-fidelity prototypes are a good compromise between time and quality, allowing you to test more specific questions, cast a vision, and communicate with stakeholders better.
- High-fidelity prototypes take the most investment in time and skills, and allow you to test fine-tooth details. They're great for selling a concept to a client, and finalizing work. They're also good for communicating final design decisions to developers so that they can implement the work.

TESTING

INTRODUCTION

Testing is part of an iterative phase of the design thinking process that provides you with feedback, based on rigorous testing of the prototype. The purpose of testing is to learn what works, and what doesn't and then iterate. This means going back to your prototype and modifying it, based on feedback from the users. Testing ensures that you come back to the essential core of design thinking - empathy of users and designing for their needs.

WHY DO WE NEED A TEST?

A test is an assessment to establish the validity, reliability and quality of design of a product or service. You can conduct tests in any form of formality as long as you are able to achieve your end objective. For example, a test could just be as simple as walking up to your neighbour with a prototype and asking a few questions. Otherwise, a test could be conducted in a more structured manner.

We test our prototypes because:

1 — So we can validate and iterate quickly 2

— Match expectations of end users

3 — Evaluate feedback

4 — To communicate results to end users

HOW TO TEST

After you've created your first, or hundredth, prototype and you want to test it, there's a bit of setup work you need to do. Testing with users is the process of getting someone who is as close to your ideal user as possible, and who is not you, to interact with your prototype to test a specific assumption or to find any pain points, problems, or confusion within your proposed idea. If the device is normally used by one person, test one person at a time. If it is normally used by a group, test a group. The only exception is that even if the normal usage is by a single person, it is useful to ask a pair of people to use it together, one person operating the prototype, the other guiding the actions and interpreting the results (aloud). Using pairs in this way causes them to discuss their ideas, hypotheses, and frustrations openly and naturally.

The research team should be observing, either by sitting behind those being tested (so as not to distract them) or by watching through video in another room (but having the video camera visible and after describing the procedure). Video recordings of the tests are often quite valuable, both for later showings to team members who could not be present and for review. In person, you can look for micro expressions of frustration, delight, and confusion, and then ask follow-up questions to why the user felt that way. If you conduct testing remotely, you can listen for any hesitations and gain nearly as much insight into the question or assumption you're testing. Both in-person and remote testing provide quantitative and qualitative data that lets you examine the intuitive use and emotional response to an experience.

When the study is over, get more detailed information about the people's thought processes by retracing their steps, reminding them of their actions, and questioning them. Sometimes it helps to show them video recordings of their activities as reminders.

How many people should be studied? Opinions vary, but Jakob Nielsen, has long championed the number five: five people studied individually. Then, study the results, refine them, and do another iteration, testing five different people. Five is usually enough to give major findings. And if you really want to test many more people, it is far more effective to do one test of five, use the results to improve the system, and then keep iterating the test-design cycle until you have tested the desired number of people. This gives multiple iterations of improvement, rather than just one.

USER TESTING METHODS

The most popular user testing techniques used by UX (User Experience) designers and design thinkers are:

1. Concept testing

In the very early stages of the design process, you'll want to test out your initial concepts before actually designing them. Low-fidelity prototypes—a simple sketch, or even static images—can be used to communicate your idea to your target users. You'll then interview your users to gauge how they feel about the concept. Is it a product or feature they'd be interested in using? Does it have the potential to solve the user's problem?

Strengths:

- Early-stage concept validation shows you which ideas won't work before you've spent too much time and money on them
- A positive response boosts your confidence in your design decisions and priorities—and you can use positive feedback to sell your ideas to stakeholders
- It's an exploratory form of testing, which means it can help you empathize with your users to understand what they do—and don't—want

Limitations:

- Since it's a conceptual form of testing, users don't engage with a product or prototype, so their response may be inaccurate
- If you don't ask very clear questions and ask users to justify their responses, you can get fuzzy results that won't help you make key design decisions

2. A/B testing

A/B testing is used to compare two different versions of a design/prototypes. This method can be used at any stage of the design process, whether you have paper prototypes or fully clickable digital ones. In A/B testing, you'll create two different prototypes and test each version on a different set of users. You might test two different layouts, for example, or different copy for a certain CTA button on a certain screen. It's important to only A/B test one variable at a time so as not to skew the results.

Strengths:

- When you've narrowed down your design options, split testing is a great way to make final decisions before iterating
- Testing different options lowers the risk factor for new design ideas
- It can be set up with your current users—you don't necessarily have to recruit focus groups

Limitations:

- A/B and multivariate tests only give you answers to very specific, variable-dependent goals
- If you're split testing with real users, you run the risk of frustrating users with an unpopular design idea. This is a bigger problem with A/B testing, where you could potentially be testing with half of your user base!

3. Usability testing

A crucial user testing method that should be used repeatedly throughout the design process, usability testing shows you how easy your design is to use. Usability testing is usually an observational exercise: you'll ask your users to complete certain tasks, and observe them as they do so (this is part of a task analysis). Throughout the test, you'll see which aspects of the design caused problems for the user, as well as which aspects appear to be user-friendly. In doing so, you'll identify usability issues which you'll seek to fix in the next iteration of your prototype.

Strengths:

- Great for sanity-checking whether your product journey and navigation hierarchy are intuitive to real users.
- Helps you quickly identify blockers, bugs, and gaps in usability.
- Shows you how your users think, move, and navigate so you can improve the user experience (UX).

Limitations:

- You often end up with complex results based on particular participants' end goals and environments—it's not a quick test you can easily statistically summarize.
- Since you're watching real participants in real time as they use the product, it can be time-consuming.

4. First-click testing

When designing an app or a website, you want to make sure that the user takes the intended action whenever they land on a certain page or screen. First-click testing shows you what your users first steps are when they encounter an interface; in other words, where do they click first? This helps you to determine which visual elements and content should take priority, where buttons, icons, and menu items should be located, as well as the kind of language you should use for buttons and labels. First-click testing can be conducted using both low and high-fidelity prototypes.

Strengths:

- It's a good way to check the perceived usefulness of a particular feature or element for users
- It shows you which buttons, icons, and other navigational elements your users are failing to notice or avoiding
- It's a flexible form of testing: you can run first-click tests on a wireframe version of your product, on each page of your website, and on your final product

Limitations:

- First-click testing only shows you where users did or didn't click—it doesn't tell you why. Without more information, you'll end up making guesses or misunderstanding their motives: maybe you think they're not clicking a new feature button because they don't see it as useful, but, in reality, it's just not well-placed and they haven't noticed it on the page.

5. Tree testing

Once you've come up with the information architecture (IA) of your digital product, you can use tree testing to see how user-friendly it really is. You'll present the user with a "tree" of information—representative of how your site menus would be laid out—and ask them to find specific items. If users struggle to locate certain information, you'll need to rethink your information architecture. Tree testing is often conducted as a remote, unmoderated study, but it can also be done in person using paper prototypes.

Strengths:

- Great way to quickly validate whether your design is creating a clear, intuitive navigation experience for your user
- Time-efficient tests that are easy to set up (they're also easy to find recruits for as you're not asking for much of their time)
- Offers actionable data on which features or site elements need to be labelled or presented differently

Limitations:

- Like with card sorting, users only interact with a basic conceptual model of the product. That means you won't get insights on how users would respond to the full product 'in the wild', with added features, visual elements, and environmental cues that may change their experience.
- Only gives basic data on which elements are blocking users —without digging deeper, you won't understand why they're struggling or what they're trying to do

Advantages, and disadvantages of user Testing/ Validation.



The goal of user testing/validation is to understand how users experience certain products or services. The usability of products and services can be validated at any stage in the design process. This method can be particularly useful to evaluate usability of concepts, prototypes, physical or software products and service offerings of an organization.

Advantages of User Testing/ Validation

1. Feedback

Getting feedback helps designers validate their efforts on a project.

2. Next Steps

Data collected can guide the next steps to be taken to improve the product or service.

3. Benchmark against existing products

Data collected can help evaluating the product features or the service being offered against similar and competitive offerings in the market.

4. Identify missed issues

Issues that may have been missed during the other stages of the project can be captured through user testing.

5. Online Tools

Through online tools a large sample can be considered for user testing. The data gathered can also be analyzed quickly and with relative ease.

Disadvantages of User Testing/ Validation

1. User time commitment

If the users do not give sufficient time for feedback or to test, then the system or product can be off from what the user requirement is.

2. Scope of the project

If the scope of the project is large, then user testing may result in large amount of diverse data may be difficult to collate and analyze.

ITERATION

It is a process of repeating a set of operations until a specific result is achieved. The role of iteration in human-centered design is to enable continual refinement and enhancement. The goal is rapid prototyping and testing, or in the words of David Kelly, Stanford professor and cofounder of the design firm IDEO, "Fail frequently, fail fast."

Deliberate tests and modifications make things better. Failures are to be encouraged—actually, they shouldn't be called failures: they should be thought of as learning experiences. If everything works perfectly, little is learned. Learning occurs when there are difficulties.

The hardest part of design is getting the requirements right, which means ensuring that the right problem is being solved, as well as that the solution is appropriate. Requirements made in the abstract are invariably wrong. Requirements produced by asking people what they need are invariably wrong. Requirements are developed by watching people in their natural environment.

Getting the requirements right involves repeated study and testing: iteration. Observe and study: decide what the problem might be, and use the results of tests to determine which parts of the design work, which don't. Then iterate through all processes once again. Collect more design research if necessary, create more ideas, develop the prototypes, and test them.

With each cycle, the tests and observations can be more targeted and more efficient. With each cycle of the iteration, the ideas become clearer, the specifications better defined, and the prototypes closer approximations to the target, the actual product. After the first few iterations, it is time to start converging upon a solution. The several different prototype ideas can be collapsed into one.

When does the process end? That is up to the product manager, who needs to deliver the highest-possible quality while meeting the schedule. In product development, schedule and cost provide very strong constraints, so it is up to the design team to meet these requirements while getting to an acceptable, high quality design.

For example, a quality software user interface is developed through a series of design iterations. Each version is reviewed and tested, and the design is then iterated based on the feedback. The interface typically progresses from low fidelity to high fidelity as more is learned about the interface and how it will be used. Iteration occurs in all development cycles in two basic forms: ***design iteration*** and ***development iteration***.

Design iteration is the expected iteration that occurs when exploring, testing, and refining design concepts. Each cycle in the design process narrows the wide range of possibilities until the design conforms to the design requirements. Prototypes of increasing fidelity are used throughout the process to test concepts and identify unknown variables. Members of the target audience should be actively involved in various stages of iterations to support testing and verify design requirements. Whether tests are deemed a success or failure is irrelevant in design iteration, since both success and failure provide important information about what does and does not work. In fact, there is often more value in failure, as valuable lessons are learned about the failure points of a design. The outcome of design iteration is a detailed and well-tested specification that can be developed into a final product.

Development iteration is the unexpected iteration that occurs when building a product. Unlike design iteration, development iteration is rework—i.e., unnecessary waste in the development cycle. Development iteration is costly and undesirable, and generally the result of either inadequate or incorrect design specifications, or poor planning and management in the development process. The unknowns associated with a design should ideally be eliminated during the design stage. One of the most effective methods of reducing development iteration is to ensure that all development members have a clear, high-level vision of the final product. This is often accomplished through well-written specifications accompanied by high-fidelity models and prototypes.

PITCHING

To get a business stakeholder to buy in and invest, you must make a prototype, test your concept, and pitch a working version of your idea. Only then, you'll be taken more seriously, and you'll be able to clearly demonstrate the benefit of your new product.

HOW TO CREATE A PITCH?

1. The first thing you'll want to articulate is the essence of your product, service, or experience. Offer context, the main thrust of your idea, why it's different, and any call to action you're making. Try to succinctly explain it in less than a minute.
2. You'll want your pitch to be clear and unambiguous, so don't get bogged down in the details. Sell your idea by sharing how and why it counts.
3. Next you'll want to get that story into some kind of format. It could be a pamphlet, a website, a book, or a presentation. You may need more than one. You may need a graphic designer or writer to help.
4. You'll likely communicate differently with different audiences. Make sure you that as you Create a Pitch you think about telling stories of varying lengths and in varying degrees of detail.

Guy Kawasaki's 10/20/30 presentation rule

Guy Kawasaki, a Silicon-Valley based author, speaker, entrepreneur, venture capitalist and brand evangelist. (Brand evangelism is when someone is so in love with your product quality, style, customer service, or values that they literally spread the good news about you every chance they get.)

The rule goes as follows:

- There should be no more than 10 slides.
- The presentation should last no more than 20 minutes.
- There should be no text smaller than 30 point font size.

Guy Kawasaki's 10/20/30 rule was designed with the business pitch in mind, but it can be applied to all kinds of presentations. This rule ensures that your presentation is legible and concise, making it more retainable, resulting in bigger wins for your team.

10 Slides

Kawasaki says that there should be no more than 10 slides. This is because the human mind can only fully comprehend 10 different points in one sitting, considering you have one distinct point for each slide.

Limiting yourself to such a small number of slides challenges you to include only the most necessary information in your presentation so that every slide counts. For example, say you're presenting on the success of your recent campaign. Your marketing strategy was likely extensive, and you took a series of different actions to obtain your end result. Instead of outlining every aspect of your campaign, you would use your slides to outline its main elements of your strategy. This could look like individual slides for summarizing the problem you hoped to solve, your goals, the steps you took to reach your goals, and post-campaign analytics data that summarizes your accomplishments.

Tips to lessen your number of slides

1. **Opt for visuals and graphics over text.**

As the popular saying goes, "A picture is worth a thousand words." Instead of blocky text, fill your slides with visual elements like photos, graphs, and charts.

2. **Have a strong and informative script.**

Your slides' main purpose is to make an impact and complement what you're saying. If you can do away with a few details and info, go ahead and remove them from your slides and include them in your script instead.

3. **Edit, edit, edit.**

With presentations, it is often true that less is more. Keep trimming and editing until your slides are lean and only include the significant parts, without any fluff included.

20 Minutes

It's normal for people to lose focus, get distracted, or run through their to-do list in their head while watching a presentation, and it has nothing to do with you or your topic. To keep your audience engaged and interested, keep it short and sweet. Regardless of the time you have blocked out for the meeting, your team should aim to keep their presentation under 20 minutes.

Kawasaki acknowledges that presentation time slots can often be longer, but finishing at the 20-minute mark leaves time for valuable discussion and Q&A. It prevents your audience from getting bored, but you also get to interact and build rapport with them. Also, saving time in your presentation also leaves space for technical difficulties.

Tips to keep your presentation under 20 minutes

1. **Divide your presentation into sections.**

It is helpful to look at your presentation as separate sections, such as intro, main body, and conclusion. If you're planning to present in under 20 minutes, you can then divide the time for each section depending on how much time you think it requires.

2. **Time yourself.**

You will never know how long your presentation will take until you actually present it. When practicing, time yourself to see whether you fit the duration, and adjust accordingly.

30 Point Font

Kawasaki's rule of thumb is to keep all text to 30-point font or bigger. Of course, the bigger the font, the less text you'll be able to fit. This is a good exercise to decide what information you really need on the slide, and what you can do without. By making your slides more legible for your audience, you're encouraging them to follow along without straining their eyes.

The 30-point-font rule helps in making your presentation more impactful, more digestible text for the audience. It also forces the presenter to not simply read the slide, but speak which allows the audience to focus on listening than seeing the slides.

Summary

Each element of the rule works in tandem with the other: limiting yourself to 10 slides requires you to select the most salient points to present to your audience. A 20-minute timeline helps you ensure that you're contextualizing those slides as you speak, without delving into unnecessary information. Using a 30-point font can act as a final check for your presentation, as it emphasizes the importance of only displaying key points on your slides, rather than huge blocks of text. Font size then circles back around to the ten slides, as you'll craft sentences from your key points that will fit on your slides in 30-point font.